



UNIVERSIDADE DO VALE DO ITAJAÍ

Escola Politécnica

Documentação do Sistema

Sistema IoT para monitoramento ambiental

Autores: Gustavo Sena Vieira, Samuel Pereira Trizotto de Andrade

Orientador: Prof. Doutor Maurício de Campos
Iotec Lab

Itajaí – SC
3 de Dezembro de 2025

Contents

Histórico de Revisões	2
1 Introdução	3
1.1 Objetivo do Sistema	3
1.2 Arquitetura de Alto Nível	3
1.3 Organização do Documento	4
2 Parte I: Hardware e Firmware	4
2.1 Especificação de Hardware	4
2.2 Esquemático e Pinagem	4
2.3 Sistema de Alimentação e Consumo	5
2.4 Firmware e Lógica Embarcada	6
2.4.1 Bibliotecas Utilizadas	6
2.4.2 Fluxo de Execução	6
2.5 Estrutura de Arquivos do Firmware (ESP-IDF)	7
2.6 Função principal (Envio de mensagem via LoRa)	7
2.7 Protocolo de Dados (Payload)	8
2.7.1 Decodificador na TTN (Payload Formatter)	8
3 Parte II: Aplicação Web	10
3.1 Introdução	10
3.2 Front-End	10
3.2.1 Página Inicial	10
3.2.2 Página Admin	12
3.3 Back-End (Golang)	15
3.3.1 Arquivo de Configuração (.env)	16
3.3.2 Módulo Principal (Main)	16
3.3.3 Package Data (Estruturas e Variáveis)	16
3.4 Package MQTT	17
3.4.1 Conexão (client.go)	17
3.4.2 Handler de Mensagens (client.go)	18
3.4.3 Exemplo de Payload JSON	18
3.4.4 Gerenciamento de Tópicos (topicos.go)	19
3.5 Package API	19
3.5.1 Rotas e Propósitos	19
3.5.2 Configuração do Servidor	20
4 Banco de Dados	21
4.1 Introdução e Recomendações	21
4.2 Collections	21
4.2.1 dataCollection	21
4.2.2 sensorsCollection	22
4.2.3 topicsCollection	22

HISTÓRICO DE REVISÕES

Versão	Data	Responsável	Descrição das Alterações
1.0	03/12/2025	Gustavo Sena Vieira	Criação inicial do documento.

1 INTRODUÇÃO

Esta documentação apresenta a especificação técnica e o manual de operação do sistema (NOME DO PROJETO). O projeto foi desenvolvido no laboratório IotecLab, e tem como propósito realizar o monitoramento ambiental remoto da região litoral de Santa Catarina, que abrange Itajaí, Navegantes, Balneário Camboriú, Penha e Balneário Piçarras.

1.1 Objetivo do Sistema

O objetivo principal é monitorar e fornecer dados em tempo real de variáveis ambientais, permitindo a análise histórica e a tomada de decisão baseada em dados.

As funcionalidades principais incluem:

- Coleta de dados sensoriais, sendo eles temperatura, umidade, pressão atmosférica em intervalos configuráveis.
- Transmissão de longa distância utilizando a tecnologia LoRa/LoRaWAN.
- Armazenamento centralizado em banco de dados na nuvem.
- Visualização de dados através de uma interface Web interativa.

1.2 Arquitetura de Alto Nível

O sistema é composto por duas grandes partes Hardware (Edge) e Software (Web). A comunicação entre elas ocorre através de um Gateway, conforme ilustrado na Figura 1.



Figure 1: Diagrama de Blocos da Arquitetura do Sistema (ou Gateway WisGate)
Fonte: O Autor.

O fluxo de dados segue o seguinte caminho:

1. **End Device (Nó Sensor):** Realiza a leitura dos sensores e transmite os pacotes de dados e os envia para o gateway utilizando o protocolo LoRaWAN.
2. **Gateway e Network Server:** O Gateway recebe o sinal de rádio e o encaminha para a The Chirpstack Network Server. A Chirpstack decodifica a mensagem e a disponibiliza através de uma integração MQTT.
3. **Back-end (Subscriber):** O servidor da aplicação atua como um cliente MQTT, assinando o tópico específico na TTN. Ao receber a mensagem, ele processa o dado e o armazena no Banco de Dados.

4. **Front-end (Dashboard):** A interface web consulta os registros no Banco de Dados e apresenta as informações visualmente (gráficos e tabelas) para o usuário.

1.3 Organização do Documento

Para facilitar a manutenção, este documento está dividido em duas partes técnicas principais:

- **Parte I - Hardware e Firmware:** Detalha os esquemáticos, componentes eletrônicos e código embarcado nos microcontroladores.
- **Parte II - Aplicação Web:** Detalha a API, banco de dados e interface do usuário.

2 PARTE I: HARDWARE E FIRMWARE

Esta seção detalha as especificações físicas do dispositivo (End Device), as conexões elétricas e a lógica do software embarcado.

2.1 Especificação de Hardware

Abaixo são listados os componentes utilizados na montagem do protótipo.

Table 1: Lista de Materiais

Componente	Descrição/Modelo	Qtd.
Microcontrolador	Heltec WiFi LoRa 32 V2	1
Sensor Ambiental	Sensor de Temperatura DHT22	1
Alimentação	Na prototipação foi utilizado o próprio notebook	1
Outros	Protoboard	1
Outros	Jumpers(Macho-Macho)	5
Outros	Resistor 4.7k(ohms)	1

2.2 Esquemático e Pinagem

A Figura 2 apresenta o diagrama de conexões elétricas do sistema.

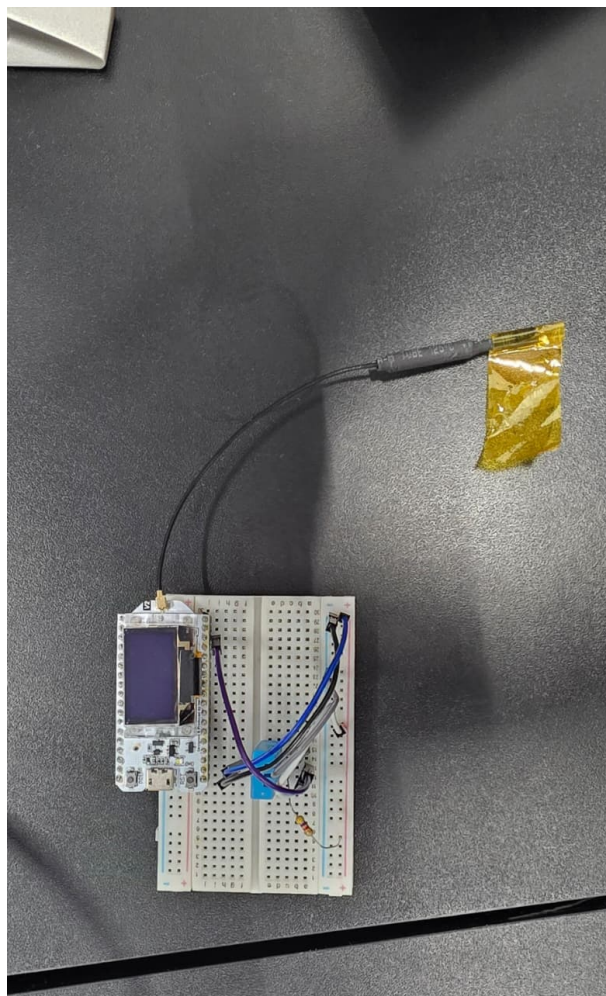


Figure 2: Esquemático Elétrico das Conexões
Fonte: O Autor.

Para facilitar a manutenção e reprodução, a Tabela 2 descreve a relação entre os pinos do microcontrolador e os periféricos.

Table 2: Pinout (Mapa de Conexões) - ESP32 e DHT11

Pino ESP32	Pino Sensor (DHT11)	Função
5V	VCC	Alimentação Positiva
GND	GND	Referência (Terra)
GPIO 4	DATA	Leitura de Dados (Temp/Umid)

OBS: Se utilizar o sensor DHT11 avulso (sem placa), é obrigatório adicionar um resistor de $4.7k\Omega$ entre o pino VCC e o pino DATA para estabilizar o sinal.

2.3 Sistema de Alimentação e Consumo

O sistema é alimentado pelo próprio notebook do autor.

- **Tensão de Operação:** 5V
- **Autonomia Estimada:** Ainda não foi testado
- **Gerenciamento de Energia:** O microcontrolador "dorme" após enviar o dado ao gateway.

2.4 Firmware e Lógica Embarcada

O firmware foi desenvolvido utilizando a linguagem C no ambiente VS Code, utilizando a extensão ESP-IDF, que é da própria desenvolvedora dos chips da esp32.

2.4.1 Bibliotecas Utilizadas

- [manuelbl/ttn-esp32]: [Para controle do LoRa]
- [zorxx/dht] (v1.0.1): [Para leitura do Sensor DHT11]

2.4.2 Fluxo de Execução

O comportamento do firmware segue o ciclo de vida abaixo (Fluxograma):

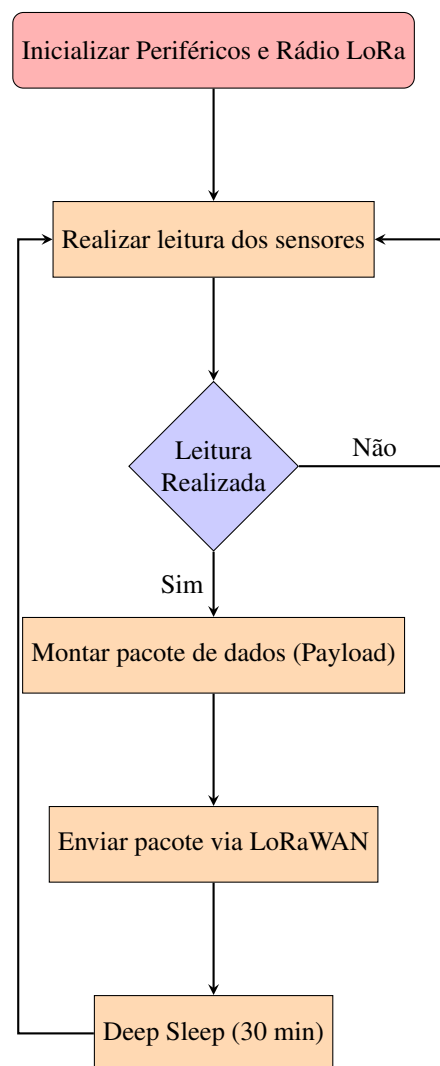


Figure 3: Fluxograma da Lógica do Firmware com Verificação de Erro

A lógica principal consiste em:

1. Inicializar periféricos e rádio LoRa.
2. Realizar leitura dos sensores.

3. OBS: Caso a leitura não seja feita, é feito 5 tentativas e se mesmo assim não for possível ler os dados, é enviado uma mensagem de erro.
4. Montar o pacote de dados/mensagem de erro (Payload).
5. Enviar pacote via LoRaWAN.
6. Entrar em modo de baixo consumo (Deep Sleep) por 30 minutos.

2.5 Estrutura de Arquivos do Firmware (ESP-IDF)

O projeto foi desenvolvido utilizando o framework oficial **ESP-IDF** (Espressif IoT Development Framework). A organização segue o modelo baseado em componentes:

- `main/`: Diretório principal da aplicação.
 - `main.c`: Ponto de entrada do firmware (função `app_main`). Contém a inicialização do hardware e o loop de tarefas.
 - `CMakeLists.txt`: Instruções de compilação para o módulo principal.
- `components/`: Componentes personalizados ou bibliotecas manuais.
 - `ttn-esp32/`: Biblioteca responsável pela conexão LoRa.
- `managed_components/`: Bibliotecas gerenciadas automaticamente pelo *IDF Component Manager*.
 - `zorxx__dht/`: Biblioteca para leitura do sensor de temperatura e umidade (DHT11/DHT22)(essa deve ser baixada no *IDF Component Manager*).
- `sdkconfig`: Arquivo gerado pelo `idf.py menuconfig`. Contém todas as configurações do projeto (SSID WiFi, credenciais LoRa, pinos, frequência da CPU). **Nota:** Este arquivo define o comportamento do hardware sem alterar o código C.

2.6 Função principal (Envio de mensagem via LoRa)

A função `sendMessages` é a tarefa principal do sistema operacional em tempo real (FreeRTOS). Ela é responsável por orquestrar a leitura do sensor e a transmissão dos dados.

O código abaixo demonstra o loop infinito que realiza a leitura dos dados de temperatura e umidade, formata o pacote (payload) em uma string CSV e o envia através da pilha LoRaWAN:

```

1 void sendMessages(void* pvParameter)
2 {
3     // Definição do tipo de sensor (DHT11 ou DHT22)
4     dht_sensor_type_t sensor_type = DHT_TYPE_DHT11;
5     float temperature, humidity;
6     // Aguarda estabilização inicial do sensor
7     vTaskDelay(pdMS_TO_TICKS(2000));
8
9     while (1) {
10         // Tenta realizar a leitura do sensor
11         if (dht_read_float_data(sensor_type, DHT_GPIO, &humidity, &temperature) ==
            ↳ ESP_OK) {
12
13             // Log no monitor serial para debug
14             printf("Temp: %.1f C | Hum: %.1f %%\n", temperature, humidity);

```

```

15
16      // Preparação do Buffer para string (payload em ASCII)
17      char payload[64];
18      snprintf(payload, sizeof(payload), "temperatura,%.1f,C,umidade,%.1f,%%",
19          ↪ temperature, humidity);
20
21      printf("Payload gerado: %s\n", payload);
22
23      // Envio via LoRaWAN
24      // O parâmetro 'l' é a porta da aplicação, 'false' é sem confirmação
25      ↪ (unconfirmed)
26      ttn_response_code_t res = ttn_transmit_message((uint8_t*)payload,
27          ↪ strlen(payload), 1, false);
28
29      printf(res == TTN_SUCCESSFUL_TRANSMISSION ? "Message sent.\n" :
30          ↪ "Transmission failed.\n");
31
32      } else {
33          printf("Failed to read from DHT sensor\n");
34      }
35      // Aguarda o intervalo definido antes da próxima leitura
36      vTaskDelay(TX_INTERVAL * pdMS_TO_TICKS(1000));
37  }
38  }

```

2.7 Protocolo de Dados (Payload)

O firmware está configurado para enviar os dados em formato de texto legível (ASCII String), facilitando a depuração e visualização direta dos dados brutos.

O formato da string gerada é:

```
"temperatura,[T.T],C,umidade,[H.H],%"
```

Exemplo de mensagem enviada:

```
temperatura,24.5,C,umidade,60.0,%
```

2.7.1 Decodificador na TTN (Payload Formatter)

Como a The Things Network recebe os dados em formato hexadecimal (bytes), é necessário um decodificador para converter esses bytes de volta para texto.

Abaixo está o código Javascript utilizado no console da Chirpstack Network Server (Device Profiles -> [NOME DO DEVICE UTILIZADO] -> Codec):

```

function decodeUplink(input) {
  return {
    data: {
      message: String.fromCharCode(...input.bytes)
    },
    warnings: [],
    errors: []
  };
}

```

}

Nota Técnica: O envio de dados em formato ASCII (Texto) consome mais tempo de transmissão (Airtime) e bateria do que formatos binários. Para versões futuras do projeto, deve ser considerada a possibilidade da migração para envio de bytes puros.

3 PARTE II: APLICAÇÃO WEB

3.1 Introdução

A aplicação final é composta por 3 módulos: Front-end em React, Back-end em Go (Golang) e o banco de dados MongoDB. O repositório está disponível em: <https://github.com/IoTec-Lab-Univali-Itajai/Site-IoTec-Sensores>.

3.2 Front-End

3.2.1 Página Inicial

O código principal da página inicial está disponível no diretório `src` (Figura 4), onde também se encontra a pasta com os componentes.

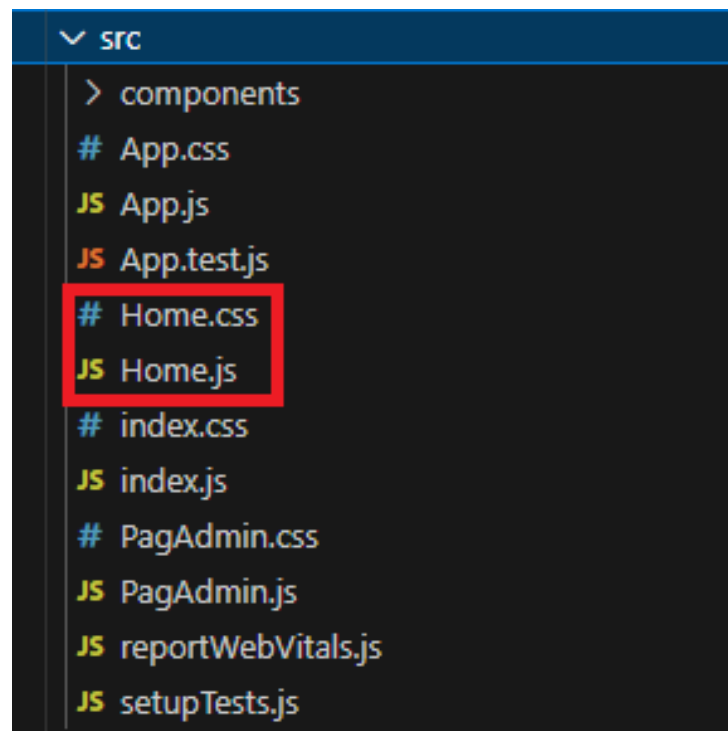


Figure 4: Diretório SRC e Componentes

A página principal envia uma requisição para o backend buscar o último dado dos sensores marcados para serem exibidos na home (explicação de backend na seção 3.5, API), conforme o seguinte código:

```
1 function Home() {
2   const [sensores, setSensores] = useState([]);
3
4   useEffect(() => {
5     const fetchSensores = async () => {
6       try {
7         const res = await fetch("http://10.1.203.113:8080/api/sensorView");
8         if (!res.ok) throw new Error("Erro ao buscar sensores ativos");
9         const data = await res.json();
10
11         // Mapeia a estrutura do backend para o que o frontend espera
12         const sensoresFormatados = data.map(sensor => ({
13           mqttID: sensor.SensorID, // SensorID do backend vira mqttID no frontend
```

```

14         dados: sensor.SensorData // SensorData do backend vira dados no frontend
15     }));
16
17     setSensores(sensoresFormatados);
18     console.log('HOME.JS DIZ: Dados recebidos:', sensoresFormatados);
19 } catch (err) {
20     console.error("Erro ao carregar sensores:", err);
21     setSensores([]);
22 }
23 };
24 // ... continuação da função

```

O backend retorna um JSON que contém uma string com nome do sensor de tag "SensorID" e um objeto com a tag "SensorData". Cada objeto "SensorData" possui:

- Uma string de tag "infoType" contendo o tipo de informação (exemplo: temperatura);
- Um float64 de tag "valor" contendo o valor (exemplo: 23,5);
- Uma string de tag "unidade" contendo o sinal da unidade (exemplo: °C).

Esse JSON é convertido em objetos e salvo no vetor/variável `sensores`, que possui vários objetos chamados `sensor`, contendo `sensor.mqttID` (SensorID) e `sensor.dados` (SensorData).

```

const sensoresFormatados = data.map(sensor => ({
    mqttID: sensor.SensorID, // SensorID do backend vira mqttID no frontend
    dados: sensor.SensorData // SensorData do backend vira dados no frontend
}));

```

A `Home.js` envia essas informações ao renderizar os componentes `CardSensor` com base no tamanho da variável `sensores`:

```

{sensores.length === 0 ? (
    <p>Nenhum sensor ativo no momento.</p>
) : (
    sensores.map(sensor => (
        <CardSensor
            key={sensor.mqttID}
            nome={sensor.mqttID}
            dados={sensor.dados || []} // Garante que dados seja array
        />
    ))
)}

```

O componente `CardSensor`, por sua vez, renderiza cada um dos blocos de informações dos sensores conforme o recebido de cada sensor (Figura 5).

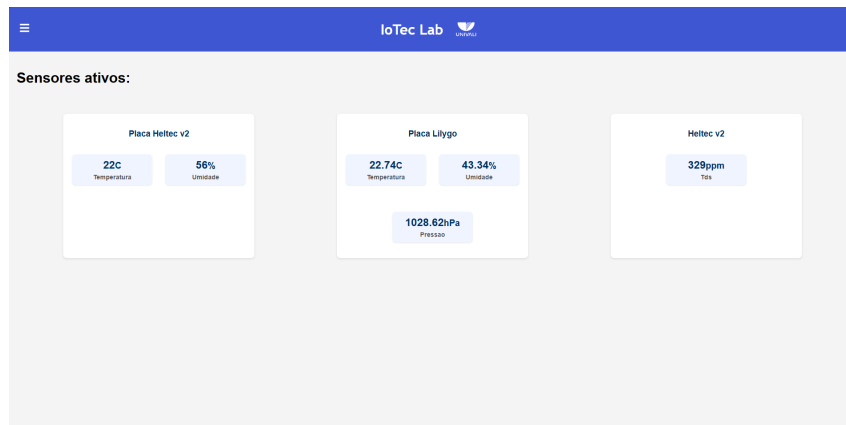


Figure 5: Exemplo de renderização do CardSensor

3.2.2 Página Admin

O código principal da página de administrador está disponível no diretório `src` (Figura 6), onde também se encontra a pasta com os componentes.

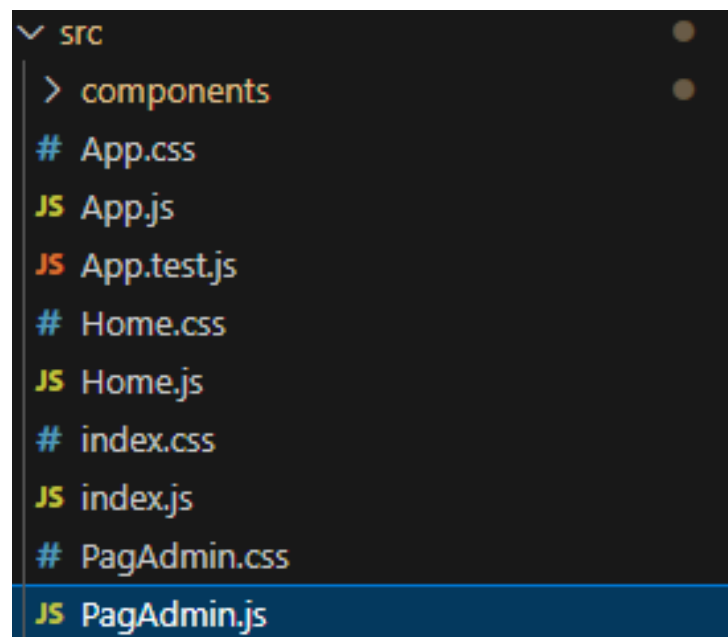


Figure 6: Interface da Página de Admin

A página de administrador possui duas funções principais:

- Registrar tópicos e sensores novos.
- Mostrar o status dos sensores, como a checkbox de exibição na home e a data da última mensagem enviada.

Ao entrar na página, o sistema envia uma requisição para o backend para obter todos os tópicos no banco de dados conforme o código:

```
function PagAdmin() {
  const [topicos, setTopicos] = useState([]);
  const [topicoSelecioneado, setTopicoSelecioneado] = useState(null);
```

```

const [sensoresSelecionados, setSensoresSelecionados] = useState([]);

// 1. Buscar tópicos ao montar
useEffect(() => {
  fetch('http://10.1.203.113:8080/api/topicsJSON')
    .then(response => response.json())
    .then(data => setTopicos(data))
    .catch(err => console.error('Erro ao buscar tópicos:', err));
}, []);

```

Ao selecionar um tópico, o sistema envia uma requisição ao backend para obter todos os sensores cadastrados naquele tópico, conforme o código:

```

useEffect(() => {
  if (!topicoSelecionado) return;

  console.log('URL completa:',
    ↪ 'http://10.1.203.113:8080/api/sensorJSON?topic=${topicoSelecionado}');
  const topicEncoded = encodeURIComponent(topicoSelecionado);
  console.log('Tópico codificado:', topicEncoded);

  fetch(`http://10.1.203.113:8080/api/sensorJSON?topic=${topicEncoded}`)
    .then(response => response.json())
    .then(data => setSensoresSelecionados(data))
    .catch(err => console.error('Erro ao buscar sensores:', err));
}, [topicoSelecionado]);

```

As informações dos sensores são recebidas em uma estrutura JSON. A estrutura abaixo (em Go) exemplifica as tags e tipos de dados utilizados na comunicação:

```

// Estrutura de dados (Backend) correspondente ao JSON recebido
type Sensor struct {
  MqttID      string    `bson:"mqttID" json:"mqttID"`
  TopicName   string    `bson:"topicName" json:"topicName"`
  Descricao   string    `bson:"descricao" json:"descricao"`
  LastUpdate  time.Time `bson:"lastUpdate" json:"lastUpdate"`
  LastData    []SensorValue `bson:"lastData" json:"lastData"`
  ShowOnScreen bool      `bson:"showOnScreen" json:"showOnScreen"`
}

```

O `PagAdmin.js` chama o componente `StatusSensor.js` uma única vez, passando as informações e funções principais conforme o código:

```

<StatusSensor
  sensores={sensoresSelecionados}
  onRemoveSensor={handleRemoverSensor}
  topico={topicoSelecionado}
  onSensorAdded={handleSensorAdded}
/>

```

O componente `StatusSensor.js` renderiza o HTML para cada sensor no array de dados recebidos do `PagAdmin`, que por sua vez recebeu esses dados do backend. Ele também renderiza o botão de adicionar sensores e a legenda das cores.

No quesito funções, `StatusSensor` chama a função de `PagAdmin` ao remover sensores. A função `handleSensorAdded` é recebida por `StatusSensor` apenas para ser passada para o `BotaoAdd`. As funções de modificação do status de exibição na home são internas do `StatusSensor`.

O componente `BotaoAdd.js` é responsável por renderizar o formulário de acordo com o que está sendo adicionado (passado no parâmetro “texto”). O processo de adicionar um tópico/sensor ocorre de forma dividida. A função da API que chama o backend para criar um tópico/sensor novo no banco de dados é chamada diretamente pelo `BotaoAdd`. Porém, as funções que atualizam os arrays do frontend com as informações dos sensores e tópicos são da `PagAdmin` (`handleAdded` e `handleSensorAdded`) e o `BotaoAdd` apenas as invoca.

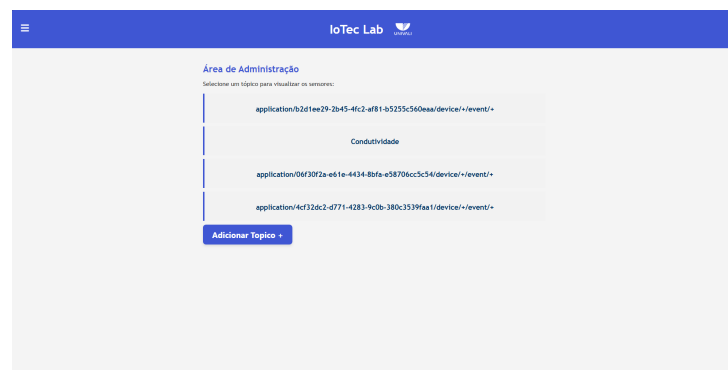


Figure 7: `PagAdmin.js` ao renderizar

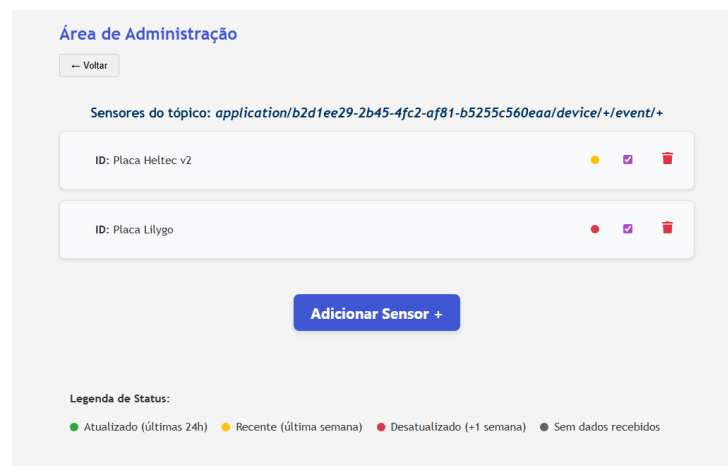


Figure 8: `PagAdmin.js` ao selecionar um tópico)

The image shows a modal window titled "Adicionar Novo Sensor". It contains three main sections: "ID MQTT:" with a text input field showing "Ex: sensor_temperatura_01"; "Descrição:" with a larger text area containing the placeholder "Descrição do sensor (opcional)"; and "Mostrar na Home:" with a dropdown menu currently set to "Sim". At the bottom right of the modal are two buttons: "Cancelar" (grey) and "Adicionar" (green). The modal is overlaid on a blurred background of a web application.

Figure 9: formulário de cadastro de sensor renderizado pelo BotaoAdd.js

3.3 Back-End (Golang)

O back-end foi feito usando Go (Golang) e está disponível no diretório `backend` conforme a Figura 10. Os nomes dos pacotes são os mesmos dos diretórios onde cada arquivo `.go` se encontra.

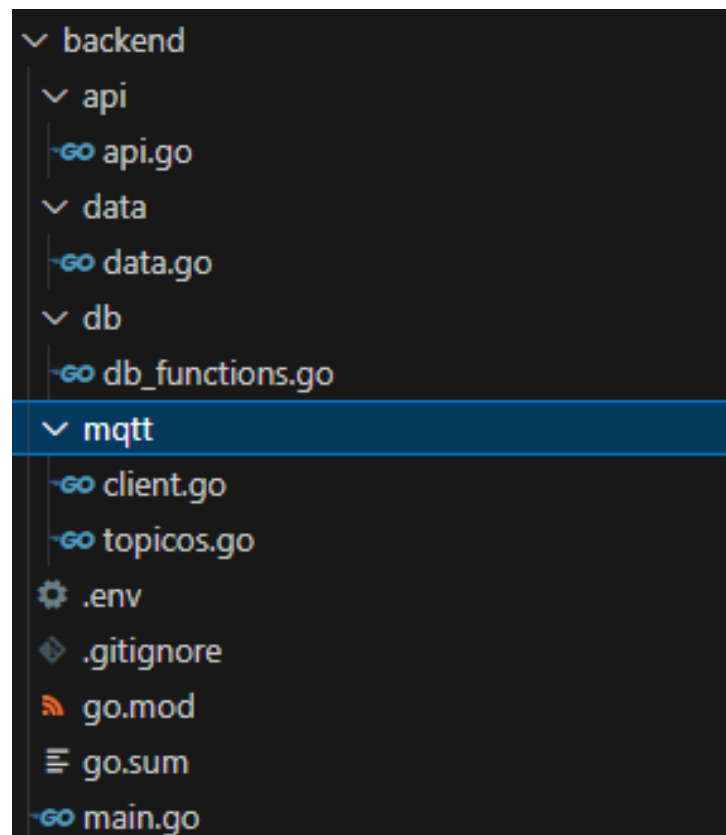


Figure 10: Diretório Backend e Estrutura de Pacotes

As principais funções do back-end incluem:

- Acessar e escutar o MQTT do Network Server da Chirpstack.
- Receber as mensagens do MQTT e enviar ao front-end conforme a variável `ShowOnScreen` de cada sensor.
- Salvar os dados recebidos pelo MQTT no Banco de Dados.
- Modificar o banco de dados conforme as ações feitas no front-end (adição, remoção e edição).
- Fornecer ao front-end as demais informações dos tópicos e sensores salvas no banco de dados.
- Gerenciar a conexão nos diferentes tópicos MQTT.

3.3.1 Arquivo de Configuração (.env)

O `.env` é um arquivo usado para configurações gerais e variáveis de ambiente. Um exemplo de `.env` utilizado no projeto é:

```
# Configurações MQTT
MQTT_BROKER_URL=000.111.22.33:1883 # ou mqtt://broker.example.com:1883
MQTT_CLIENT_ID=meuID123
MQTT_USERNAME=meuUsuario123
MQTT_PASSWORD=minhaSenha123

# Configurações do Banco de dados
# Exemplo local ou Atlas
MONGO_HOST=mongodb://localhost:27017/
# ou mongodb+srv://meuusuario:minhasenha123@cluster0.abc123.mongodb.net/...
```

Por padrão a API se comunica com o frontend na porta `IPdoServidorBackend:8080`. Tanto o IP do servidor quanto a porta podem ser configuradas via `.env` (necessário modificar o código para puxar essa informação). No código exemplo, tanto no front quanto no back-end, os endereços foram escritos estaticamente no código e devem ser trocados manualmente em caso de mudança de ambiente.

3.3.2 Módulo Principal (Main)

O código `main` é conciso e tem como função apenas iniciar as conexões gerais (carregar `.env`, conectar ao banco de dados, conectar ao MQTT) e iniciar a API (comunicação com o frontend).

```
func main() {
    err := godotenv.Load("./.env")
    if err != nil {
        log.Println("Main.go diz: Arquivo .env não encontrado, prosseguindo sem
        ↪ ele")
    }

    db.ConnectMongoDB()

    mqtt.BuscarConectarTopicos();

    api.StartAPI() // Inicia servidor em :8080
}
```

3.3.3 Package Data (Estruturas e Variáveis)

O pacote `data` foi criado com o propósito de armazenar as `Structs` e as variáveis globais utilizadas por todos os arquivos `.go`, evitando a dependência cíclica e importação desnecessária de `packages`.

Funções que alteram as variáveis também devem ser adicionadas nesse arquivo para manter a coerência. No projeto exemplo, como cada arquivo realiza operações bem específicas nas variáveis globais, não foi necessária a criação de funções acessórias no package data.

A lista abaixo descreve as principais estruturas e variáveis:

Struct Topic Uma struct com fins de organização, criada apenas para melhor leitura de código. Facilita futuras alterações na definição dos tópicos (ex: adicionar uma descrição).

Struct Sensor É a struct do sensor cadastrado na `SensorsCollection` (seção 4.2.2) no banco de dados, com todos os requisitos mínimos. É a estrutura enviada ao front-end no menu de Admin.

Struct SensorData É a struct dos dados salvos na `DataCollection` (seção 4.2.1) no banco de dados.

Struct SensorValue É a struct que armazena os valores recebidos no payload do MQTT. Cada instância representa um tipo de valor (Exemplo: "temperatura, 23, °C"). É utilizada como componente no objeto `LastData` do sensor.

Struct InfoDisplay É a struct enviada para ser exibida na home do frontend. É composta por todos os `SensorValue` enviados por um sensor mais o ID dele no sistema.

Var DadosDisplay É a lista (slice) com todos os `InfoDisplay` que são exibidos no frontend. É utilizada para atualizar os dados em tempo real dos sensores marcados como `ShowOnScreen = true`.

Var DadosDisplayMutex É a variável de semáforo (Mutex) usada para gerenciar a concorrência de acesso à lista `DadosDisplay`, garantindo a integridade dos dados quando múltiplas threads acessam a variável simultaneamente.

3.4 Package MQTT

O package MQTT é composto por dois arquivos: `client.go`, que define a função de conexão ao MQTT e o handler de mensagens recebidas; e `topicos.go`, que define a função de buscar os tópicos e conectar ao MQTT, gerenciando reinicializações quando novos tópicos são adicionados.

3.4.1 Conexão (client.go)

A função `ConnectMQTT(topicosString string)` é responsável por receber a string com os tópicos separados por “ ” (espaço) e fazer a conexão com o MQTT, utilizando as configurações do `.env`:

```
// Configurações do broker via env
broker := os.Getenv("MQTT_BROKER_URL")
clientID := os.Getenv("MQTT_CLIENT_ID")
username := os.Getenv("MQTT_USERNAME")
password := os.Getenv("MQTT_PASSWORD")

opts.AddBroker(broker)
opts.SetClientID(clientID)
opts.SetUsername(username)
opts.SetPassword(password)
opts.SetCleanSession(true)
opts.SetProtocolVersion(4)
opts.SetAutoReconnect(true)
opts.SetConnectRetry(true)
opts.SetConnectRetryInterval(5 * time.Second)
opts.SetDefaultPublishHandler(messageHandler)
```

É importante lembrar que cada MQTT possui requisitos diferentes de conexão e, portanto, essa parte pode precisar ser alterada. O programa exemplo deste artigo acessa o MQTT de um Network Server da Chirpstack, com login e senha. Ao lidar com a Chirpstack, os tópicos para escutar todos os eventos de todos os dispositivos seguem o formato:

application/[application ID]/device/+/event/+

Se o MQTT utilizado aceitar espaços (" ") no nome do tópico, a string de tópicos não pode usar espaço como separador.

3.4.2 Handler de Mensagens (client.go)

O handler lida com as mensagens recebidas e dispara uma thread (goroutine) para cada mensagem, isolando o processamento.

```
var messageHandler mqtt.MessageHandler = func(client mqtt.Client, msg mqtt.Message) {
```

A sequência das ações feitas pelo handler inclui:

1. Mostrar no console a mensagem recebida.
2. Extrair o ID do dispositivo remetente e as informações mandadas.
3. Parsear as informações recebidas (Função ParseSensorPayload).
4. Atualizar as informações na lista de sensores exibidos na Home (Data Package).
5. Atualizar `lastData` e `lastUpdate` se o sensor estiver cadastrado no BD.
6. Inserir o dado na coleção de dados históricos se o sensor estiver cadastrado.

3.4.3 Exemplo de Payload JSON

Abaixo, um exemplo de mensagem recebida do Chirpstack. Note que a mensagem real do sensor está dentro do campo `object.message`.

```
{
  "deduplicationId": "valorGenerico",
  "time": "2025-09-26T21:26:27.476+00:00",
  "deviceInfo": {
    "tenantId": "valorGenerico",
    "tenantName": "valorGenerico",
    "applicationId": "valorGenerico",
    "applicationName": "valorGenerico",
    "deviceProfileId": "valorGenerico",
    "deviceProfileName": "valorGenerico",
    "deviceName": "ID DO SENSOR",
    "devEui": "valorGenerico",
    "deviceClassEnabled": "valorGenerico",
    "tags": {}
  },
  "devAddr": "valorGenerico",
  "adr": true,
  "dr": 5,
  "fCnt": 46,
  "fPort": 1,
  "confirmed": false,
  "data": "valorGenerico",
  "object": {
    "message": "temperatura,24.0,C,umidade,47.0,%"
  },
  "rxInfo": [
    {
      "gatewayId": "valorGenerico",
      "uplinkId": 0000000000,
      "gwTime": "valorGenerico",
      "nsTime": "valorGenerico",
      "timeSinceGpsEpoch": "valorGenerico",
      "rssi": -00,
      "snr": 00.0,
      "channel": 0,
      "rfChain": 0,
      "location": {
        "latitude": -00.00000,
        "longitude": -00.00000,

```

```

        "altitude": 00,
        "source": "valorGenerico"
    },
    "context": "valorGenerico"
}
],
"txInfo": {
    "frequency": 000000000,
    "modulation": {
        "loras": {
            "bandwidth": 000000,
            "spreadingFactor": 0,
            "codeRate": "valorGenerico"
        }
    }
},
"regionConfigId": "valorGenerico"
}

```

Neste caso, o payload decodificado é: temperatura, 24.0, C, umidade, 47.0, %.

3.4.4 Gerenciamento de Tópicos (topicos.go)

A função `BuscarConectarTopicos()` realiza a busca e conexão inicial:

1. Utiliza funções do DB para buscar todos os tópicos salvos.
2. Carrega esses tópicos para a memória.
3. Desconecta (caso já esteja conectado, para evitar duplicidade ao atualizar).
4. Conecta aos tópicos MQTT carregados.

Esta função é chamada no `main` e sempre que um tópico novo é criado via API.

3.5 Package API

O package API é composto pelo arquivo `api.go` e é responsável pela conexão entre frontend e backend. Os métodos de requisição são:

- **GET**: Requisição de busca de informações do backend para o front.
- **POST**: Requisição para atualizar informações do front para o backend.
- **PUT**: Atualizar informações do front para o backend.
- **DELETE**: Excluir algum dado do backend.
- **OPTIONS**: Busca informações sobre o protocolo de comunicação (não utilizado no sistema exemplo).

3.5.1 Rotas e Propósitos

As rotas e seus propósitos são:

`r.Get("/api/topicsJSON", GetTopicsJSON)`

Busca os tópicos no banco de dados e retorna em formato JSON para o frontend.

`r.Get("/api/sensorJSON", GetSensorsJSON)`

Busca os sensores pertencentes a um tópico (selecionado no frontend) no banco de dados e retorna em formato JSON para o frontend.

`r.Get("/api/sensorView", GetSensorView)`

Retorna todos os sensores salvos em `DadosDisplay` (seção 3.3.3 Package Data) no formato JSON para o frontend.

`r.Delete("/api/sensorDelete/{id}", DeleteSensor)`

Deleta um sensor de ID específico do banco de dados e também das variáveis de sistema (como `DadosDisplay`).

r.Post("/api/sensorCreate", CreateSensor)

Cria um sensor novo no banco de dados e nas variáveis de sistema (como DadosDisplay) com informações enviadas pelo frontend.

r.Post("/api/topicCreate", CreateTopic)

Cria um tópico novo no banco de dados com informações enviadas pelo frontend e refaz as conexões MQTT com o tópico novo incluso.

r.Put("/api/sensorUpdate/{id}", UpdateSensorVisibility)

Atualiza a visibilidade dos sensores no banco de dados e a existência dele na variável DadosDisplay (seção 3.3.3 Package Data).

3.5.2 Configuração do Servidor

No sistema exemplo, a comunicação entre front e back é feita através da porta 8080. O backend está configurado para permitir requisições com origem na porta 3000 (frontend).

As configurações adicionais (CORS e Router) são:

```
r := chi.NewRouter()
r.Use(middleware.Logger)
r.Use(cors.Handler(cors.Options{
    AllowedOrigins:  []string{"http://00.0.000.000:3000"},
    AllowedMethods:  []string{"GET", "POST", "PUT", "DELETE", "OPTIONS"},
    AllowedHeaders:  []string{"Accept", "Authorization", "Content-Type",
        ↪ "X-CSRF-Token"},
    ExposedHeaders:  []string{"Link"},
    AllowCredentials: false,
    MaxAge:          300, // cache preflight for 5 minutes
}))

// ... (Definição das rotas listadas acima) ...

http.ListenAndServe(":8080", r)
```

4 BANCO DE DADOS

4.1 Introdução e Recomendações

O banco de dados utilizado pelo sistema exemplo foi o MongoDB, um banco de dados NoSQL que armazena documentos. Seu modo de operação é ideal para lidar com grandes quantidades de dados que chegam em formatos variados. Cada coleção tem suas regras de campos obrigatórios no sistema, mas permitem boa flexibilidade no formato dos documentos. Não é recomendado o uso de banco de dados relacionais (SQL) para esse tipo de aplicação. Caso o volume de acessos esteja causando lentidão no sistema, é possível implementar índices no banco de dados para agilizar as consultas e bancos em memória como REDIS para servir de cachê. Para melhor monitoramento do banco de dados, recomenda-se utilizar a plataforma MongoDB Compass. Todos os links de download dos sistemas citados estão disponíveis nas referências.

4.2 Collections

No MongoDB os documentos são organizados em collections (coleções). O sistema exemplo utiliza 3 collections:

- **dataCollection:** Salva os dados recebidos dos sensores.
- **sensorsCollection:** Salva os sensores cadastrados.
- **topicsCollection:** Salva os tópicos cadastrados.

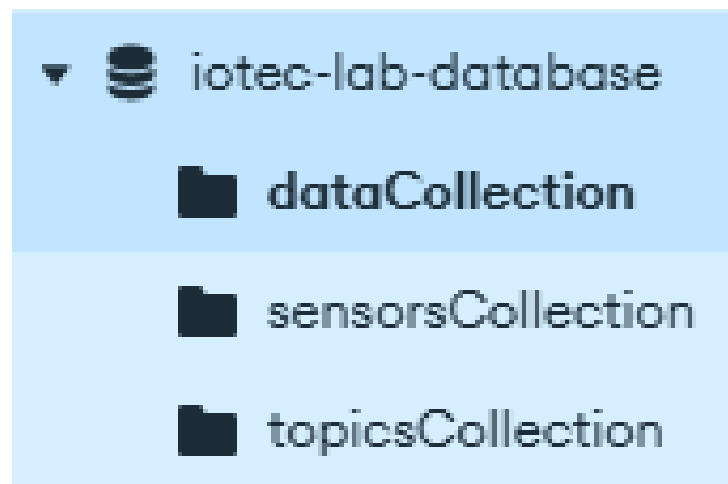


Figure 11: collections dentro do database iotec-lab-database. Captura de tela da plataforma MongoDB Compass

A relação entre as collections é feita através de dados iguais em determinados campos e será explicada mais adiante.

4.2.1 dataCollection

Responsável por guardar os dados recebidos dos sensores, a dataCollection possui a seguinte estrutura:

- **String sensorId:** Equivalente ao campo mqttID na sensorsCollection. Corresponde ao ID cadastrado do sensor pelo frontend, no network server e na mensagem mqtt.
- **Date timestamp:** A data em que o dado foi salvo no banco de dados.
- **Object dados:** Um objeto de estrutura variada que possui os dados enviados pelo sensor. É equivalente à um conjunto de instâncias da estrutura SensorValue (seção 3.3.3 Package Data).

```

    _id: ObjectId('68e82d02a000224896c0c67b')
  ▾ dados : Object
    ▾ temperatura : Object
      valor : 23
      unidade : "C"
    ☒ umidade : Object
      valor : 62
      unidade : "%"
    sensorId : "Placa Heltec v2"
    timestamp : 2025-10-09T21:45:38.791+00:00

```

Figure 12: Estrutura de um documento da dataCollection na plataforma MongoDB Compass

4.2.2 sensorsCollection

Responsável por guardar os cadastros dos sensores, a sensorsCollection possui a seguinte estrutura:

- **String mqttID:** É equivalente ao nome cadastrado no NetworkServer e no campo deviceName na mensagem MQTT. É o nome dado ao dispositivo por quem o cadastrou.
- **String topicName:** Armazena o nome do tópico ao qual o sensor pertence. Equivalente ao campo nome na topicsCollection.
- **String descricao:** Uma descrição breve opcional para o dispositivo.
- **Date lastUpdate:** A data do último dado enviado pelo sensor.
- **Object Array lastData:** Um array de objetos de estrutura variada que possui os últimos dados enviados pelo sensor. É equivalente à um conjunto de instâncias da estrutura SensorValue (seção 3.3.3 Package Data).
- **Bool showOnScreen:** Armazena o valor de true para exibir os dados do dispositivo na home e false para ocultar o dispositivo. Não interfere na captura e armazenamento dos dados.

```

    _id: ObjectId('68caf9eccdcdf2102b0a5ac5')
    mqttID : "Placa Heltec v2"
    topicName : "application/b2d1ee29-2b45-4fc2-af81-b5255c560eaa/device/+/event/+"
    descricao : "ooooooooo"
    lastUpdate : 2025-10-21T22:49:00.479+00:00
  ▾ lastData : Array (2)
    ▾ 0: Object
      infotype : "temperatura"
      valor : 24
      unidade : "C"
    ▾ 1: Object
      infotype : "umidade"
      valor : 38
      unidade : "%"
    showOnScreen : true

```

Figure 13: Estrutura de um documento da sensorsCollection na plataforma MongoDB Compass

4.2.3 topicsCollection

Responsável por guardar os tópicos cadastrados no sistema, a topicsCollection possui a seguinte estrutura:

- **String nome:** Armazena o nome do tópico. No sistema exemplo, um tópico MQTT é composto por uma estrutura fixa + dev eui da aplicação no NetworkService.

```

    _id: ObjectId('68c8567029cb7eac81f21bc7')
    nome : "application/b2d1ee29-2b45-4fc2-af81-b5255c560eaa/device/+/event/+"

```

Figure 14: Estrutura de um documento da topicsCollection na plataforma MongoDB Compass

O nome é definido da seguinte maneira:

```
application/DEV EUI no NetworkServer/device/+/event/+
```

Os sinais de “+” são valores que incluem “todos” automaticamente. Exemplo, na parte de device, caso o usuário deseje ouvir apenas um dispositivo, ele deve colocar a dev eui do dispositivo. O sinal de “+” inclui todos os dispositivos automaticamente. Quanto aos eventos, existem diferentes tipos, como Join, Up, Log. Caso o usuário deseje ouvir apenas um tipo de evento (Join para detectar dispositivos novos ou Up para ver apenas os dados transmitidos) basta trocar o “+” pelos eventos específicos.